



Benutzerdokumentation
Mühlespiel

28. Mai 2025

Bellini Silvio, Mikus Ryan, Sutter
Christian

Änderungsnachweis

Autor: Bellini Silvio, Mikus Ryan, Sutter Christian
Erstellt am: 28. Mai 2025

Version	Änderungsgrund
1	Erstellung der Dokumentation

Kurz-Z.	Datum
01	28.05.25

Inhaltsverzeichnis

1.	Einleitung	1
1.1	Zweck	1
1.2	Spielbeschreibung	1
1.2.1	Spielregeln	1
2.	Systemübersicht	2
2.1	Architektur	2
2.2	Verwendete Entwurfsmuster	3
2.2.1	MVC	3
2.2.2	Strategy	4
2.2.3	Singleton	4
2.2.4	Memento	5
2.2.5	State	5
3.	Bedienung	6
3.1	Aufstarten	6
3.2	Benutzeroberfläche	6
3.2.1	Aufbau und Navigation	6
4.	Künstliche Intelligenz (KI)	16
4.1	CPUPlayer und ImaginativePlayer	16
4.2	DifficultyStrategy	16
5.	Anhang	I
5.1	Übersicht Architektur	I

1. Einleitung

1.1 Zweck

Diese Dokumentation dient als umfassende Anleitung zur Benutzung und zum Verständnis der entwickelten Mühlespiel-Software. Sie dient als Hilfestellung für die Endnutzer zur Bedienung des Spiels und zur ausführlicheren Beschreibung des Aufbaus und der Funktionsweise des gesamten Programms.

1.2 Spielbeschreibung

Mit dem Programm kann das klassische Brettspiel «Mühle» entweder allein gegen eine KI mit wählbarem Schwierigkeitsgrad oder lokal gegen einen anderen Spieler gespielt werden. Die graphische Benutzeroberfläche (GUI) unterstützt die intuitive Bedienung.

1.2.1 Spielregeln

Das Spiel beginnt mit der **Setzphase**. Jeder Spieler verfügt über insgesamt neun Steine, die sie abwechselnd auf die 24 Felder des Spielbretts setzen. Wird eine Reihe aus drei gleichfarbigen Steinen gebildet – horizontal oder vertikal auf einer durchgehenden Linie – entsteht eine Mühle. Für jede gebildete Mühle darf der betreffende Spieler einen gegnerischen Stein entfernen. Dabei gilt: Es dürfen nur Steine entfernt werden, die nicht Teil einer Mühle sind – es sei denn, der Gegner hat ausschließlich Steine in Mühlen.

Sind alle Steine platziert, beginnt die **Ziehphase**. In dieser Phase dürfen die Spieler ihre gesetzten Steine auf benachbarte, über Linien verbundene Felder ziehen. Ziel bleibt es, durch das Verschieben neue Mühlen zu bilden, um weitere gegnerische Steine zu entfernen.

Hat ein Spieler nur noch drei Steine auf dem Spielfeld, tritt für ihn die **Springphase** ein. Ab diesem Zeitpunkt darf der Spieler beliebige freie Felder mit seinen Steinen erreichen – also unabhängig davon, ob die Felder benachbart sind. Verliert ein Spieler einen weiteren Stein und besitzt nur noch zwei, ist er nicht mehr in der Lage, eine Mühle zu bilden und verliert das Spiel.

Ein Spiel kann auf vier verschiedene Arten enden:

1. Ein Spieler hat nur noch zwei Steine – er verliert das Spiel.
2. Ein Spieler kann keinen gültigen Zug mehr ausführen – er ist blockiert und verliert. Dieses Szenario tritt nur in der Ziehphase auf.
3. Die vorgegebene Zugzeit läuft ab – der Spieler, der gerade am Zug war, verliert.
4. In 20 aufeinanderfolgenden Zügen wurde keine Mühle gebildet – das Spiel endet unentschieden. Diese Regel gilt erst ab der Ziehphase.

2. Systemübersicht

2.1 Architektur

Die Anwendung basiert auf einer modularen Schichtenarchitektur, die insbesondere das Model-View-Controller (MVC)-Entwurfsmuster zur Trennung von Logik, Benutzeroberfläche und Steuerung nutzt. Dadurch wird eine klare Zuordnung von Verantwortlichkeiten und eine gute Erweiterbarkeit des Systems gewährleistet.

Neben MVC kommen weitere Design-Patterns zum Einsatz, etwa das Strategie-Pattern zur Auswahl von KI-Strategien oder das Memento-Pattern, um Spielzüge wieder rückgängig zu machen. Die Manager-Klassen übernehmen jeweils zentrale Funktionen und koordinieren diese für die Verwendung an anderen Orten. Darunter befinden sich zum Beispiel der BoardManager, welcher die Logik des Spielfelds enthält oder der RuleManager, der darauf achtet, ob gültige Züge gemacht werden oder das Spiel zu Ende ist.

Die Software kann grob in sechs Pakete unterteilt werden:

- **Boardlogik (boardLogic):**
Kümmert sich um die Logik des Spiels, also um die Datenstruktur für das Spielbrett und die Steine oder kontrolliert, ob die Regeln nicht verletzt werden.
- **Spieler (players):**
Es gibt unterschiedliche Arten von Spielern, nämlich menschliche, die KI und fiktive Spieler. Dazu gibt es eine gemeinsame Oberklasse, die PlayerEntity, welche allgemeine Methoden für die einzelnen Spieler bereitstellt.
- **Phasen (phases):**
Jede Phase des Spiels wird von einer Klasse repräsentiert, wobei sie von einer gemeinsamen Oberklasse ableiten. Sie behandeln die Spielzüge unterschiedlich und können jeweils prüfen, ob in die nächste Phase gewechselt werden kann.
- **Schwierigkeitsgrade (difficulties):**
Die KI hat Zugriff auf unterschiedliche Schwierigkeitsgrade. Jeder davon wird von einer eigenen Klasse repräsentiert, wobei diese von einer gemeinsamen Oberklasse ableiten. Diese stellt viele Taktiken bereit, die zur Berechnung der Spielzüge verwendet werden können.
- **Darstellung (view):**
Darin befinden sich Klassen, die sich auf die Visualisierung des Spielgeschehens beschränken. Im Zentrum steht die GUI-Klasse, welche viele einzelne Klassen verwendet, um die unterschiedlichen Screens zu generieren und darzustellen. Zudem gibt es eine GUIBoard-Klasse, die sich um die visuelle Logik, beispielsweise Benutzereingaben, kümmert und bei jeder Änderung aktualisiert wird. Dies geschieht mittels Observer.
- **Speicherung (persistence):**
Zur Speicherung einzelner Züge oder des gesamten Spiels gibt es gekapselte Klassen, welche primär dazu dienen, die notwendigen Variablen auf sich zu speichern. Sie besitzen keine grossen Funktionalitäten, sondern werden lediglich «verwendet». Der GamePersistenceManager speichert mittels dieser Klassen den Momentanzustand mit allen relevanten Informationen in eine JSON-Datei und kann diese bei Bedarf wieder laden. Die History speichert die einzelnen Spielzüge, damit diese rückgängig gemacht oder wiederholt werden können. Auch sie wird in der JSON-Datei abgespeichert, um ihre Funktionalität auch nach dem Laden beizubehalten.

Im Anhang findet sich eine Übersicht der Architektur und Pakete.

2.2 Verwendete Entwurfsmuster

2.2.1 MVC

Modell (Model):

Das Modell bildet das Herzstück der Anwendung und umfasst sämtliche Spiellogik sowie den aktuellen Spielzustand. Es ist für die Regelprüfung, Spielfeldbewertung und Zugverarbeitung verantwortlich. In der vorliegenden Implementierung besteht das Modell im Wesentlichen aus zwei zentralen Klassen:

- **BoardManager:**
Diese Klasse verwaltet das Spielbrett und die Positionen der Spielsteine. Sie bietet Funktionen zur Platzierung, Bewegung und Entfernung von Steinen und prüft, ob gültige Züge vorliegen. Auch das Erkennen von Mühlen (drei in einer Reihe) gehört zu ihren Aufgaben.
- **RuleManager:**
Diese Komponente kapselt die Spielregeln und deren Interpretation. Sie entscheidet beispielsweise, ob ein Zug erlaubt ist, ob ein Spieler blockiert ist oder ob das Spielende erreicht wurde. RuleManager wird vom GameManager als Entscheidungsinstanz genutzt.

Gemeinsam bilden BoardManager und RuleManager das Regel- und Zustandsmodell des Spiels. Sie sind unabhängig von der grafischen Oberfläche und können deshalb gut getestet und wiederverwendet werden.

Steuerung (Controller):

Der Controller ist durch die Klasse GameManager repräsentiert. Diese Komponente steuert den Spielablauf und übernimmt die Vermittlung zwischen Benutzerinteraktionen (über die View) und der Spiellogik (im Modell). Zu seinen Aufgaben zählen:

- Übergabe von Benutzereingaben an das Modell
- Koordination von KI-Zügen
- Aktualisierung der View nach jeder Spielaktion

Eine zentrale Aufgabe des GameManager ist es, bei einem Spielzug auf die Benutzereingabe aus dem GUI zu warten. Erst wenn der Benutzer im GUIBoard eine gültige Eingabe getätigt hat, wird der Zug verarbeitet und das Modell entsprechend angepasst. Danach erfolgt eine Rückmeldung an die View zur Aktualisierung der Anzeige.

Durch diese Mittlerrolle ist der GameManager vollständig von der konkreten grafischen Darstellung entkoppelt – er „weiß“ nichts über Layout oder Benutzerführung, sondern konzentriert sich ausschließlich auf den korrekten Ablauf der Spielmechanik.

Ansicht (View):

Die View besteht im Projekt aus den Klassen GUI und GUIBoard, welche die Benutzeroberfläche darstellen. Ihre Aufgaben sind:

- Darstellung des aktuellen Spielzustands (Spielbrett, Steine, Mitteilungen)
- Interaktive Benutzerführung durch Schaltflächen und visuelle Rückmeldungen
- Erfassung von Benutzeraktionen (z. B. Mausklicks auf Buttons, Drag and Drop)

Die Klasse GUI initialisiert und strukturiert die gesamte Oberfläche. GUIBoard ist dabei für die konkrete Darstellung des Spielbretts zuständig und verarbeitet die direkte Interaktion mit diesem. Die View-Komponenten kommunizieren mit dem GameManager.

Die View enthält keine Spiellogik. Sie sendet Benutzeraktionen an den Controller und reagiert auf Statusänderungen, indem sie die grafische Darstellung entsprechend anpasst.

2.2.2 Strategy

Das Strategie-Pattern wird im Projekt verwendet, um das Verhalten der Künstlichen Intelligenz (KI) zur Spielentscheidung flexibel und austauschbar zu gestalten. Ziel des Musters ist es, eine gemeinsame Schnittstelle für verschiedene Strategien bereitzustellen, sodass der konkrete Algorithmus zur Zugauswahl zur Laufzeit bestimmt und leicht ausgetauscht werden kann.

Im Mühlespiel sollen Spieler die Möglichkeit haben, gegen unterschiedlich starke KI-Gegner anzutreten. Diese weisen grundsätzliche Unterschiede in ihrem Entscheidungsverhalten auf. Durch das Strategie-Pattern lassen sich diese verschiedenen Vorgehensweisen klar voneinander trennen und gezielt einsetzen, sowie einfach mit anderen Algorithmen erweitern. Diese bietet eine hohe Flexibilität für die Weiterentwicklung des Mühlespiels.

Struktur:

Im Projekt wird das Pattern wie folgt umgesetzt:

- DifficultyStrategy** (Abstrakte Basisklasse)
 Diese Schnittstelle definiert die gemeinsamen Methoden, mit denen ein Zug ermittelt bzw. ausgewählt werden kann. Alle konkreten Strategien müssen die Methoden zur Berechnung eines Zuges oder der Entfernung eines Steines implementieren. Zusätzlich haben sie Zugriff auf weitere Methoden, die alle verwenden können, z.B. einen zufälligen Zug auszuführen.
- Konkrete Strategien:**
 Diese Klassen implementieren das Verhalten für verschiedene Schwierigkeitsstufen:
 - EasyDifficulty: Führt einfache oder zufällige Züge aus. Keine strategische Planung.
 - MediumDifficulty: Bewertet Züge auf Basis einfacher Heuristiken (z. B. mögliche Mühlenbildung oder Vermeidung).
 - HardDifficulty: Nutzt einen komplexeren Entscheidungsbaum, in welchem zusätzlich einige mögliche Strategien des Gegners frühzeitig verhindert werden oder eigene vorteilhafte Spielzüge geplant werden. Ein Beispiel ist das Bilden und Verwenden einer Pendelmühle.
- CPUPlayer**
 Diese Klasse repräsentiert eine KI-gesteuerte Spielfigur. Sie verwendet ein DifficultyStrategy-Objekt, um in jeder Spielsituation den nächsten Zug zu berechnen. Die konkrete Strategie wird dabei beim Erstellen des Spielers auf einen Standardwert gesetzt und kann einfach durch eine andere Implementierung ersetzt werden, indem in der View der Schwierigkeitsgrad über einen Button ausgewählt wird.

2.2.3 Singleton

Das Singleton-Pattern wird im Projekt verwendet, um sicherzustellen, dass von bestimmten Klassen, insbesondere Managerklassen, nur eine einzige Instanz existiert und diese zentral zugänglich ist. Dies dient sowohl der Ressourcenkontrolle als auch der einheitlichen Zustandsverwaltung.

In der Anwendung wird das Singleton-Pattern vor allem bei Managerklassen eingesetzt, z. B.:

- RuleManager
- BoardManager

Diese Klassen kapseln Kernfunktionen der Spiellogik, die im gesamten System konsistent und gemeinsam nutzbar sein müssen. Das Singleton stellt sicher, dass immer dieselbe Instanz verwendet wird und Zustandsinformationen zentral gehalten werden (z. B. aktuelles Spielfeld, erlaubte Züge). Singletons sollten vorsichtig eingesetzt werden. Sie vereinfachen den Zugriff, erhöhen jedoch die Kopplung und erschweren bei übermäßiger Nutzung z. B. parallele Tests oder Modularisierung. In diesem Projekt sind sie sinnvoll, da sie auf klar abgegrenzte, globale Verantwortlichkeiten (Spielfeldverwaltung, Regelwerk) beschränkt bleiben.

2.2.4 Memento

Das Memento-Pattern wird im Projekt verwendet, um den Spielverlauf rekonstruierbar zu machen – insbesondere für Undo- und Redo-Funktionen. Es dient dazu, vergangene Zustände des Spiels zu speichern, rückgängig zu machen oder wiederherzustellen.

Bei jedem durchgeführten Spielzug wird ein Move-Objekt erzeugt, das alle notwendigen Informationen enthält, um diesen Zug eindeutig zu beschreiben. Dieses Objekt wird auf einen Undo-Stack gelegt und bildet so eine Art Schnappschuss („Memento“) des Spielverlaufs.

Ein Zug kann über die View mittels zwei Buttons rückgängig gemacht oder wiederhergestellt werden. Dies wird über den GameManager gesteuert. Wird ein Zug rückgängig gemacht, erfolgt folgender Ablauf:

1. Der zuletzt gespeicherte Zug wird vom Undo-Stack entfernt („pop“).
2. Der zuvor entfernte Zug wird auf den Redo-Stack verschoben, um ihn später erneut ausführen zu können (Redo).
3. Der nun oberste Zug auf dem Undo-Stack wird bestimmt („peek“) – dieser stellt den gewünschten Wiederherstellungszustand dar und wird erneut auf dem Spielfeld ausgeführt.

Diese zweistufige Wiederherstellung – Pop, dann Peek – ist notwendig, weil der zuletzt gespeicherte Zug den aktuellen Zustand beschreibt, nicht den Zustand vor diesem Zug. Um exakt einen Zug „zurückzugehen“, muss also auf den vorletzten Zustand zurückgesetzt werden.

2.2.5 State

Das State-Pattern wird im Projekt zur Verwaltung der Spielphasen verwendet. Es dient dazu, das Verhalten des Spiels abhängig vom aktuellen Zustand zu variieren, ohne große bedingte Logik in der Hauptsteuerung unterbringen zu müssen.

Das Spiel durchläuft verschiedene Spielphasen, die nacheinander aktiv werden:

1. Setzphase – Spieler platzieren ihre Steine.
2. Ziehphase – Spieler bewegen ihre Steine auf benachbarte Felder.
3. Springphase – Spieler mit nur noch drei Steinen dürfen auf beliebige freie Felder springen.
4. Gameoverphase – Spieler hat verloren und kann keinen Zug mehr tätigen.

Diese Phasen leiten von einer abstrakten Oberklasse GamePhase ab, sind jeweils durch eigene Klassen modelliert und kapseln das spezifische Verhalten. Nach jedem durchgeführten Zug wird die Methode nextPhase() aufgerufen. Diese Methode überprüft, ob Bedingungen vorliegen, unter denen die Phase gewechselt werden muss, z. B.:

- a) Alle Steine wurden gesetzt → Wechsel in die Ziehphase
- b) Ein Spieler hat nur noch drei Steine → Wechsel in die Springphase

3. Bedienung

3.1 Aufstarten

Um das Mühle-Spiel zu starten, muss der Java-Sourcecode zunächst kompiliert und ausgeführt werden. Zusätzlich wird die externe Bibliothek GSON benötigt, um beispielsweise Spieldaten zu speichern oder zu laden.

Voraussetzungen:

- Java Development Kit (JDK) ist installiert (empfohlen: JDK 17 oder höher).
- Die GSON-Bibliothek wurde heruntergeladen und referenziert.

3.2 Benutzeroberfläche

Die grafische Benutzeroberfläche basiert auf einem CardLayout, mit dem verschiedene "Screens" (Ansichten) dynamisch angezeigt werden. Die Navigation erfolgt über Buttons. So wird der Benutzer Schritt für Schritt durch die Spielkonfiguration bis hin zum Spiel selbst geführt.

3.2.1 Aufbau und Navigation

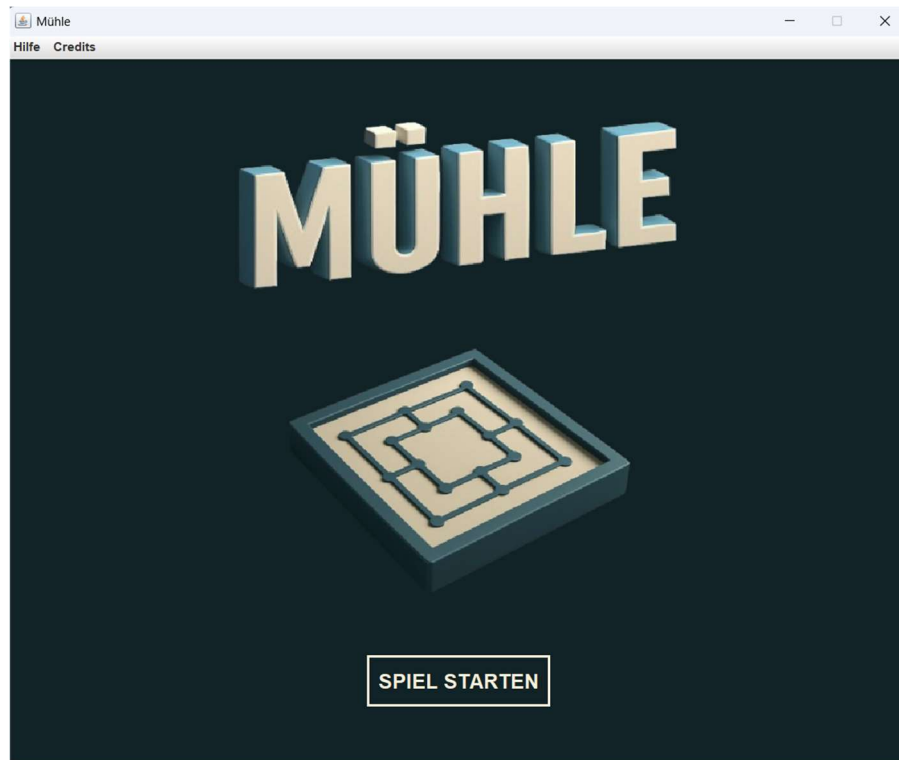
Die Anwendung besteht aus den folgenden Haupt-Screens:

- Hauptmenü
- Menü
- Spielmodus
- Schwierigkeitsauswahl
- Spielbildschirm

Die Anwendung enthält folgende Pop-Up Screens:

- GameOver
- Spiel laden
- Spiel speichern

3.2.1.1 Hauptmenü Screen

**Zweck:**

Das Hauptmenü ist der erste Bildschirm, der nach dem Starten des Spiels angezeigt wird. Es dient als Einstiegspunkt in das Spiel und bietet dem Nutzer Zugang zum Start des Spiels sowie zu Informationen über das Spiel (Credits, Hilfe).

Bedienung:

- Der Benutzer kann das Spiel mit einem Klick auf **SPIEL STARTEN** beginnen.
- Über die Menüleiste oben kann er sich vorab über das Spiel oder die Entwickler informieren.

Navigation:

Aktion	Nächster Screen
SPIEL STARTEN	Weiterleitung zum Menü
Hilfe	Öffnet die Regelübersicht
Credits	Zeigt Informationen über Mitwirkende

3.2.1.2 Menü Screen



Zweck:

Dieser Screen bietet dem Benutzer die Möglichkeit, entweder ein neues Spiel zu starten, ein gespeichertes Spiel zu laden oder zurück ins Hauptmenü zu kehren. Er dient als zentraler Ausgangspunkt für den eigentlichen Spielablauf.

Bedienung:

- Der Benutzer wählt per Mausklick eine der drei Optionen:
 - **NEUES SPIEL** startet den Spielstart-Dialog.
 - **SPIEL LADEN** ruft eine Ladefunktion auf, bei der ein gespeicherter Zustand geladen wird.
 - **ZURÜCK** führt zurück zum Startbildschirm.

Navigation:

Aktion	Nächster Screen
NEUES SPIEL	Weiterleitung zum Spielmodus
SPIEL LADEN	Öffnet Ordnerverzeichnis zur Auswahl gespeicherter Spiele (siehe Abschnitt 3.2.1.9)
ZURÜCK	Zurück zum Hauptmenü

3.2.1.3 Spielmodus Screen



Zweck:

Dieser Screen ermöglicht dem Benutzer die Wahl zwischen einem **Einzelspieler-Modus** (gegen eine KI) und einem **Mehrspieler-Modus** (zwei Menschen am selben Gerät). Zudem kann der Benutzer zum vorherigen Menü zurückkehren.

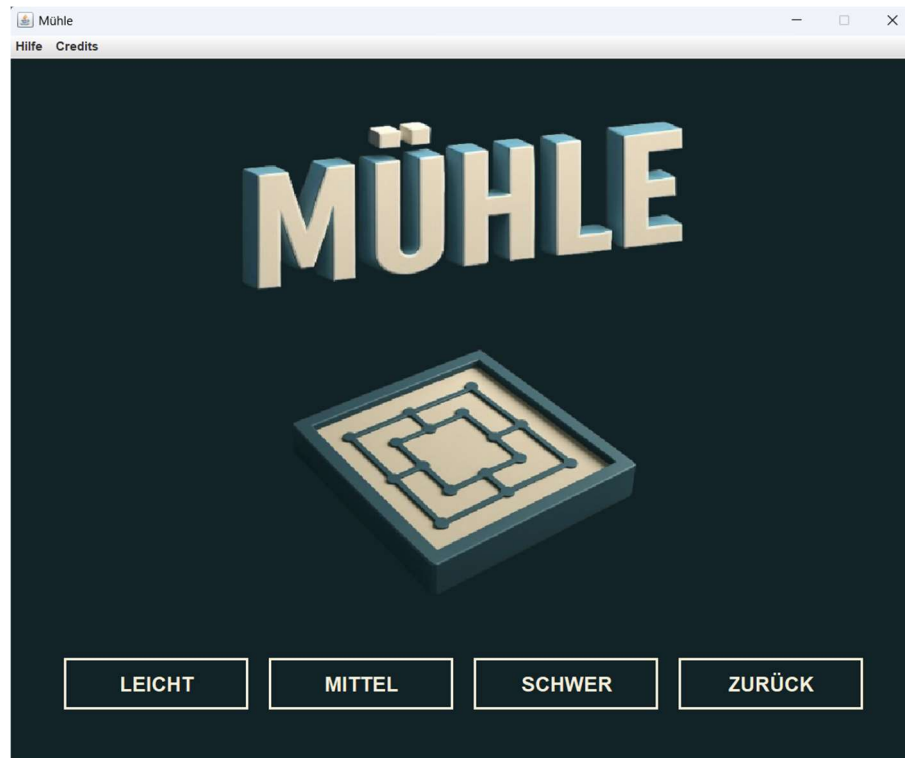
Bedienung:

- Der Benutzer wählt per Mausklick eine der drei Optionen:
 - **EINZELSPIELER** führt zum nächsten Screen (Schwierigkeitsauswahl der KI).
 - **MEHRSPIELER** beginnt das Spiel mit zwei Spielern.
 - **ZURÜCK** führt zurück zum **Menü**.

Navigation:

Aktion	Nächster Screen
EINZELSPIELER	Weiterleitung zur Schwierigkeitsauswahl
MEHRSPIELER	Direkt zum Spielbildschirm
ZURÜCK	Zurück zu Menü

3.2.1.5 Schwierigkeitsauswahl Screen



Zweck:

Dieser Screen dient dazu, im Einzelspielermodus die gewünschte Schwierigkeitsstufe der KI festzulegen. Dadurch kann sich der Spieler auf ein angemessenes Spielniveau einstellen – von leicht bis herausfordernd.

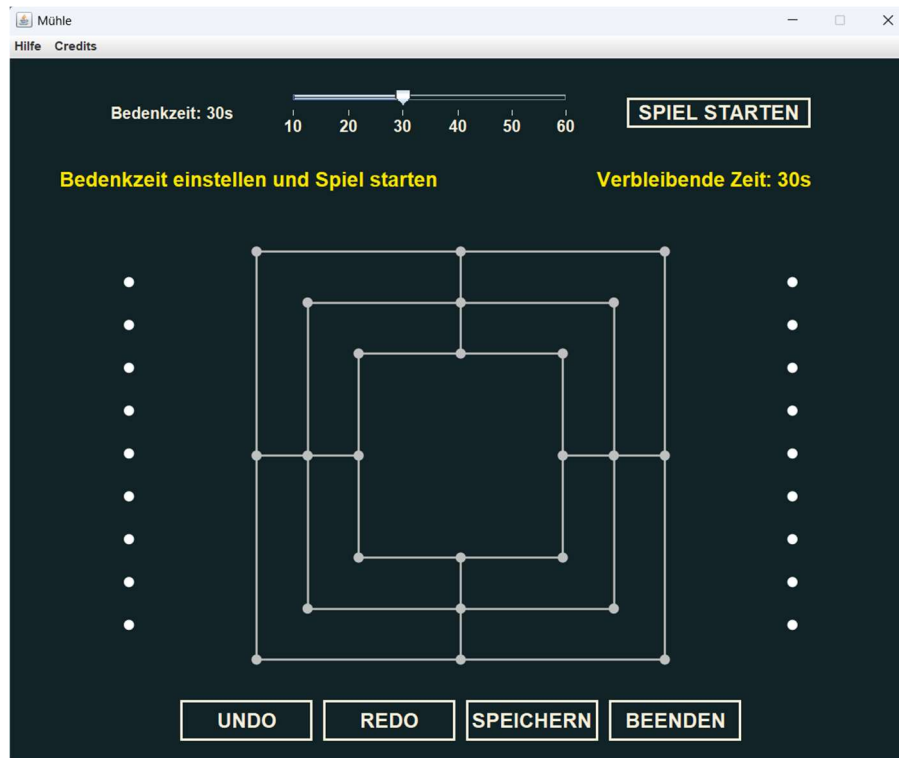
Bedienung:

- Der Benutzer wählt eine der drei Schwierigkeitsstufen durch Klick auf:
 - **LEICHT** für Anfänger oder schnelle Partien
 - **MITTEL** für fortgeschrittene Spieler
 - **SCHWER** für erfahrene Spieler mit strategischem Anspruch
- **ZURÜCK** bringt den Benutzer zurück zur Spielmodusauswahl (Einzel-/Mehrspieler)

Navigation:

Aktion	Nächster Screen
LEICHT/MITTEL/SCHWER	Direkt zum Spielbildschirm
ZURÜCK	Zurück zu Spielmodus

3.2.1.7 Spielbildschirm Screen



Zweck:

Dies ist der zentrale Spielscreen, auf dem die eigentliche Partie Mühle stattfindet. Hier wird das Spielbrett angezeigt, Züge werden ausgeführt, der Spielstand verwaltet und Optionen wie Spielstand speichern oder Zug rückgängig machen genutzt. Oben links befinden sich für den Spieler hilfreiche Anweisungen, was zurzeit gemacht werden muss, beispielsweise welcher Spieler gerade am Zug ist, oder Fehlermeldungen, falls ein ungültiger Zug versucht wurde.

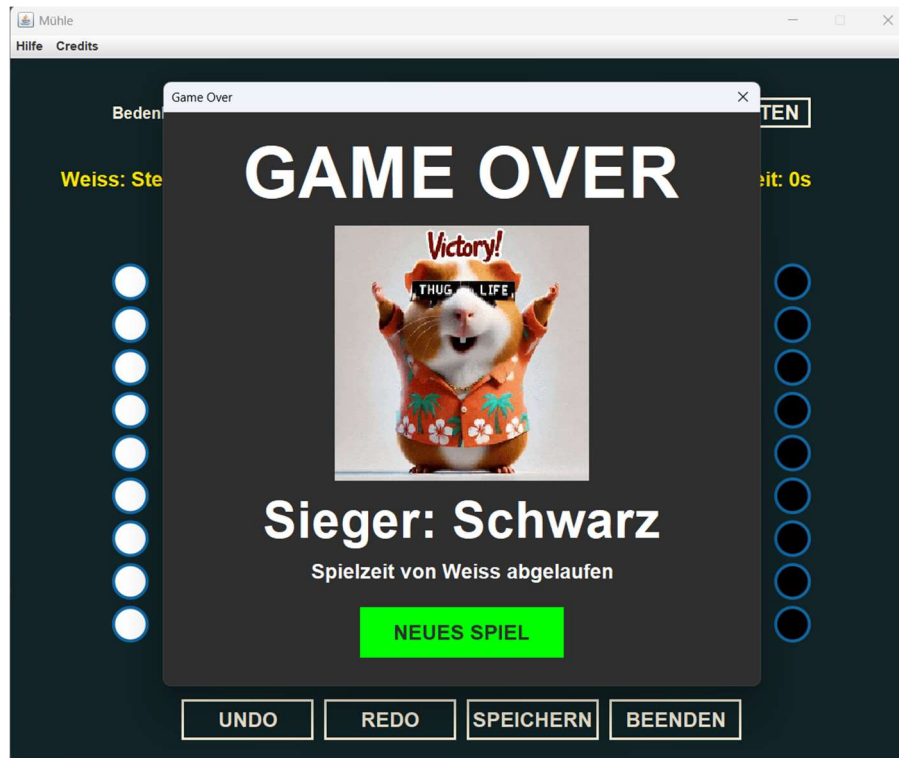
Bedienung:

- Zu Beginn kann die **Bedenkzeit pro Zug** mithilfe eines Sliders zwischen 10 s und 60 s gewählt werden.
- Mit Klick auf **SPIEL STARTEN** beginnt das Spiel und der Timer beginnt.
- Während des Spiels stehen folgende Schaltflächen zur Verfügung:
 - **UNDO** macht den letzten Zug rückgängig. Wenn gegen die KI gespielt wird, wird dessen letzter Zug ebenfalls rückgängig gemacht.
 - **REDO** stellt einen rückgängig gemachten Zug wieder her. Wenn gegen die KI gespielt wird dessen rückgängig gemachter Zug ebenfalls wiederhergestellt.
 - **SPEICHERN** speichert den aktuellen Spielstand. Währenddessen wird das Spiel pausiert und es kann nach dem Speichern direkt fortgeführt werden.
 - **BEENDEN** beendet das Spiel und kehrt ins Menü zurück, wo man den Spielmodus erneut auswählen kann.
- Die **verbleibende Zeit** pro Zug wird oben rechts angezeigt.

Navigation:

Aktion	Nächster Screen
SPIEL STARTEN	Startet das Spiel mit eingestellter Zeit
UNDO/REDO	Schrittweise Navigation durch Spielzüge
SPEICHERN	Spielstand wird gespeichert (siehe Abschnitt 3.2.1.10)
BEENDEN	Rückkehr zum Spielmodus

3.2.1.8 GameOver Screen



Zweck:

Dieser Screen informiert die Spieler über das Ende der Partie und zeigt den Sieger sowie die Ursache für das Spielende. Zusätzlich gibt er die Möglichkeit, sofort ein neues Spiel zu starten. Wird es geschlossen, kann nochmals das Spielbrett angezeigt werden, um eine neue Runde zu starten, muss in diesem Fall allerdings auf **BEENDEN** im Spielbildschirm Screen geklickt werden

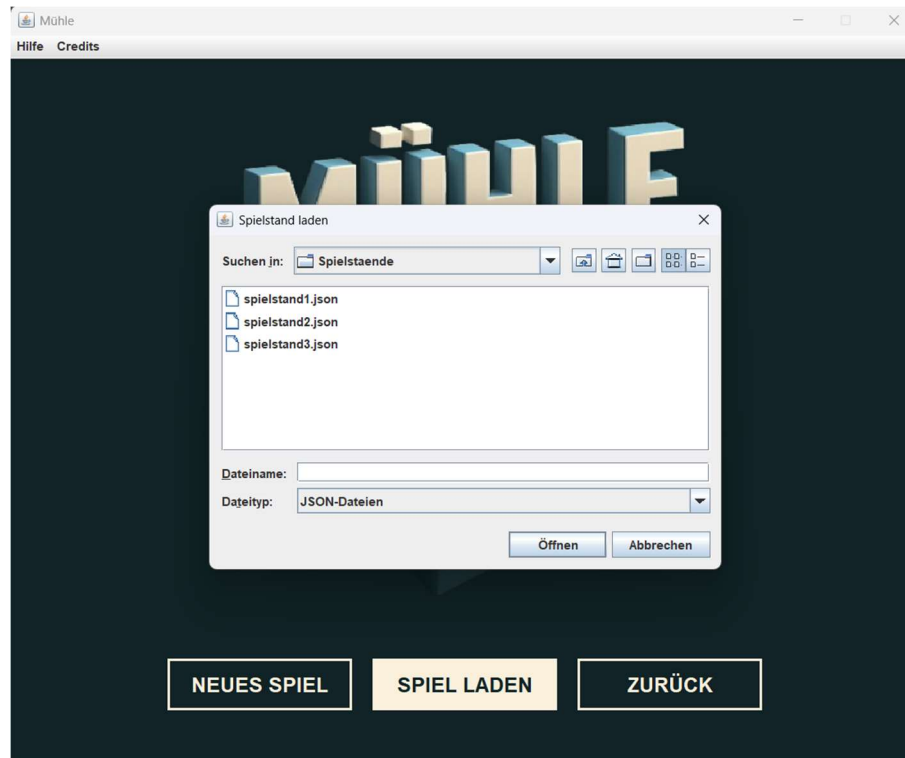
Bedienung:

- Nach Spielende erscheint ein Overlay mit:
 - Dem **Siegernamen** (z. B. „*Sieger: Schwarz*“)
 - Der **Begründung** für das Spielende (z. B. „*Spielzeit von Weiss abgelaufen*“)
 - Einem humorvollen **GIF-Element**
- Durch Klick auf den Button **NEUES SPIEL** wird ein neues Spiel gestartet (führt zurück zum Spielbildschirm mit dem ausgewählten Spielmodus und ggf. Schwierigkeitsstufe der KI des vorangegangenen Spiels). Die Bedenkzeit kann wieder neu gewählt werden.
- Der Hintergrund des Spielfelds bleibt weiterhin sichtbar, aber das Spiel ist deaktiviert.

Navigation:

Aktion	Nächster Screen
NEUES SPIEL	Zurück zum Spielbildschirm
X (Exit)	Schliesst das Fenster (nicht empfohlen für einen Neustart des Spiels)

3.2.1.9 Spiel laden Screen



Zweck:

Dieser Screen dient dazu, einen zuvor gespeicherten Spielstand auszuwählen und wiederherzustellen. Dadurch kann der Benutzer eine frühere Partie fortsetzen. Es gilt zu beachten, dass das Spiel nach dem Laden automatisch direkt weiterläuft.

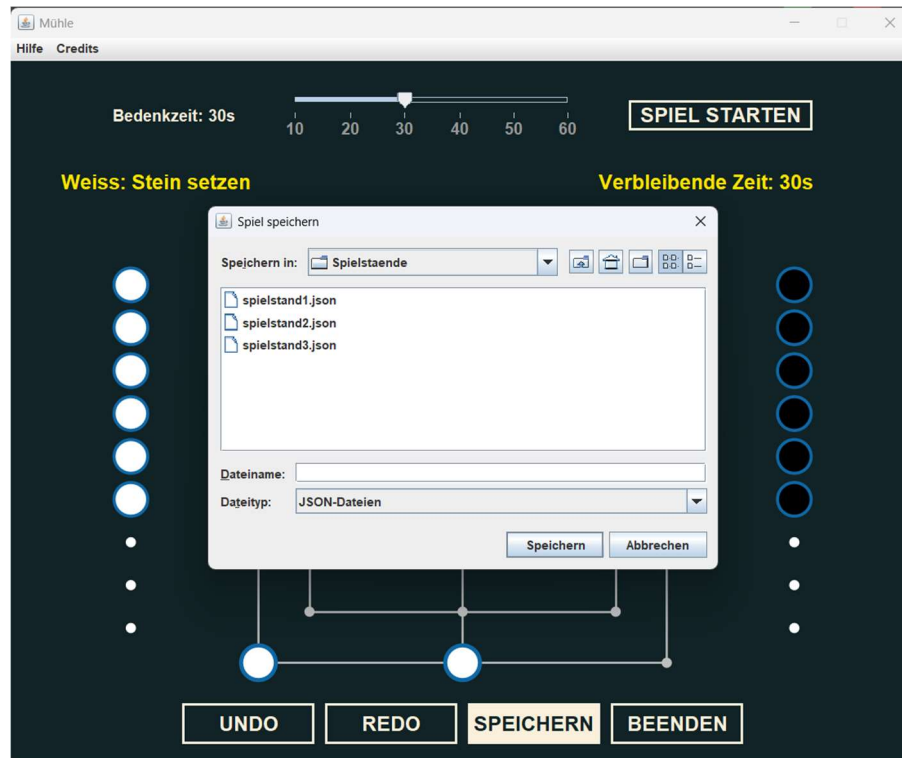
Bedienung:

- Nach Klick auf **SPIEL LADEN** öffnet sich ein Datei-Dialog.
- Der Benutzer wählt eine **.json-Datei** aus dem vorgesehenen Speicherort (z. B. spielstand1.json).
- Nach Auswahl und Klick auf **Öffnen** wird der entsprechende Spielstand geladen und das Spiel setzt sich an der letzten gespeicherten Stelle fort.
- Über **Abbrechen** kann der Ladevorgang ohne Änderung abgebrochen werden.

Navigation:

Aktion	Nächster Screen
Datei öffnen	Lädt den Spielstand und zeigt den Spielbildschirm
Abbrechen	Rückkehr zum Spielmodus

3.2.1.10 Spiel speichern Screen



Zweck:

Dieser Screen ermöglicht es dem Benutzer, den aktuellen Spielstand zu sichern, um das Spiel zu einem späteren Zeitpunkt fortzusetzen. Der Spielstand wird als **.json-Datei** abgespeichert.

Bedienung:

- Der Benutzer kann einen bestehenden Dateinamen auswählen oder einen neuen Namen eingeben.
- Mit Klick auf **Speichern** wird der aktuelle Zustand des Spiels als JSON-Datei im momentanen Ordner gespeichert.
- Über **Abbrechen** kann der Speichervorgang abgebrochen werden, ohne Änderungen vorzunehmen.

Navigation:

Aktion	Nächster Screen
Speichern	Rückkehr zum Spielbildschirm, Spiel wird fortgesetzt
Abbrechen	Rückkehr zum Spielbildschirm, ohne speichern

4. Künstliche Intelligenz (KI)

4.1 CPUPlayer und ImaginativePlayer

Die Software stellt einen KI-Gegner im Einzelspieler-Modus zur Verfügung. Der KI-Gegner hat drei Schwierigkeitsgrade – leicht, mittel und schwer. Der KI-Gegner wird im Spiel durch ein Objekt der Klasse CPUPlayer (von PlayerEntity erweitert) repräsentiert. Der Schwierigkeitsgrad wird vor dem Spiel gewählt.

Nach einem Zug des Spielers wird der CPUPlayer aufgefordert einen Zug zu machen. Dieser wartet dann 800ms, berechnet einen Zug basierend auf seinem Schwierigkeitsgrad und macht diesen. Formt er dabei eine Mühle wartet er weitere 400ms, berechnet basierend auf seinem Schwierigkeitsgrad einen Stein des Gegners, den er entfernen will und macht dies dann.

Bei den Berechnungen erstellt das CPUPlayer-Objekt zwei Objekte der Klasse ImaginativePlayer – von sich und vom Spieler. Dies dient der klaren Trennung von den Spielzügen und den Spielzugsimulationen. Die tatsächlichen Spielzugberechnungen geschehen im Objekt einer Klasse erweitert aus DifficultyStrategy. Hier sind die Zugberechnungen und Schwierigkeitsgrade zu finden.

4.2 DifficultyStrategy

Die abstrakte Klasse DifficultyStrategy hat zwei abstrakte Methoden. Eine Methode dient der Zugberechnung und eine dient der Steinentfernungsberechnung. In den Implementierungen dieser Methoden wird folgendermassen vorgegangen:

a) Zugberechnung:

Der KI-Gegner sucht sich alle möglichen Züge, die er machen kann. Anschliessend filtert er sequenziell durch seine Taktiken. Diese Taktiken sind in DifficultyStrategy zu finden. Die Reihenfolge der Taktiken zeigt die Hierarchie. Findet er bei einer Taktik keinen Zug, der dieser Taktik entspricht, schreitet er bei der nächsten Taktik fort. Damit ein Zug garantiert ist, sucht er sich beim Versagen aller Filter einen zufälligen Zug aus.

b) Steinentfernungsberechnung:

Der KI-Gegner hat hier nur zwei Taktiken – eine Taktik, die den gefährlichsten Stein des Gegners berechnet und entfernt, und eine Taktik, die einen zufälligen Stein des Gegners entfernt.

Die Klassen EasyDifficulty, MediumDifficulty und HardDifficulty sind die Klassen, wo die Taktiken sequenziell aufgerufen werden. Die Taktiken können je nach Spielphase des KI-Gegners oder Anzahl gelegter Steine die Berechnung überspringen und direkt in die nächste Taktik springen. Folgend wird dargestellt, welche Zugberechnungstaktik in welcher Phase bei welchem Schwierigkeitsgrad in welcher Reihenfolge verwendet wird:

Taktik	Verwendet in Phase	EasyDifficulty	MediumDifficulty	HardDifficulty
Pendelmühle verwenden	Ziehphase	-	-	1
Mühle machen	Alle Phasen	-	1	2
Mühle blockieren	Alle Phasen	-	2	3
Zwickmühle finden	Setzphase	-	-	4
Zwickmühle blockieren	Setzphase	-	-	5
Eckensymmetrie	Ziehphase	-	-	6
Stein in der Nähe setzen	Alle Phasen	-	3	7
Mühle öffnen	Zieh- & Sprungphase	-	-	8
Zufälliger Spielzug	Alle Phasen	1	4	9

Die Steinentfernungsberechnung ist einfacher. Bei EasyDifficulty entfernt der KI-Gegner einen zufälligen Stein. Bei MediumDifficulty und HardDifficulty entfernt der KI-Gegner einen Stein, der die grösste Bedrohung zur Mühlenbildung darstellt. Falls der KI-Gegner keinen solchen Stein findet, wird ein zufälliger Stein entfernt. Folgend werden die Taktiken genauer beschrieben:

Taktik	Beschreibung
Pendelmühle verwenden	Hat der KI-Gegner eine Mühle und kann mittels Ziehens eines dieser Steine eine andere Mühle gebildet werden, liegt eine Pendelmühle vor. Der KI-Gegner kann mit diesem Stein zwischen Mühlen pendeln.
Mühle machen	Wenn möglich, positioniert der KI-Gegner einen Stein so, dass es eine Mühle für ihn bildet.
Mühle blockieren	Wenn möglich, positioniert der KI-Gegner einen Stein so, dass es eine Mühle des Spielers blockiert.
Zwickmühle finden	Der KI-Gegner sucht eine Positionsmöglichkeit, die ihm zwei potenzielle Mühlen ermöglicht. So kann der Spieler nur eine blockieren.
Zwickmühle blockieren	Der KI-Gegner sucht eine Positionsmöglichkeit, die dem Spieler zwei potenzielle Mühlen ermöglicht, um sie zu blockieren.
Eckensymmetrie	Ist ein «konzentrischer Ring» des Spielfeldes leer und der Spieler legt einen Stein in eine Ecke dieses Ringes, soll der KI-Gegner den Stein in die entgegengesetzte Ecke legen. So verhindert der KI-Gegner die Möglichkeit auf zwei Zwickmühlen im entsprechenden Ring.
Stein in der Nähe setzen	Der KI-Gegner sucht einen Spielzug, der einen seiner eigenen Steine in der Nähe und freie Plätze als Nachbar hat. Er sucht <i>einen Nachbarn eines möglichen Zuges</i> , überprüft, ob dessen Nachbarn von mindestens zwei seiner eigenen Steine besetzt sind und mindestens einen leeren Nachbarn hat. Ist dies der Fall positioniert der KI-Gegner den Stein auf <i>den Nachbarn des möglichen Zuges</i> , sofern dieser Zug auch möglich ist.
Mühle öffnen	Der KI-Gegner öffnet seine Mühle, damit er sie wieder schliessen kann.
Zufälliger Spielzug	Der KI-Gegner wählt einen zufälligen, möglichen Zug aus.
Mühlenbedrohung entfernen	Der KI-Gegner identifiziert den strategisch wertvollsten Stein des Gegners. Strategisch wertvoll heisst, dass damit einfach eine Mühle gebildet werden kann.
Zufälliger Stein entfernen	Der KI-Gegner entfernt einen zufälligen Stein des Gegners.

5. Anhang

5.1 Übersicht Architektur

